

PlanSys2

Symbolic Task Planning for ROS 2

Middleware for Software Design in Robotics

Learning Objectives

After this lecture you should be able to:

- Understand the role of symbolic planning in robotics
- Explain the architecture of PlanSys2
- Describe how planning differs from execution
- Understand the interaction between PDDL and ROS 2
- Explain Behavior Tree execution
- Understand multi-robot execution in PlanSys2
- Design planning-based robotic applications

Why Task Planning?

Two fundamental questions in robotics:

Navigation	Planning
How do I reach the goal?	What should I do?
Low-level path finding	High-level task sequencing
Operates on geometry	Operates on logic & goals

Planning operates at a **higher level** than navigation.

Examples:

- Deliver an object
- Prepare a meal
- Guide a visitor

Classical Robot Control

Typical approaches:

- **Finite State Machines (FSM)**
- **Behavior Trees (BT)**
- **Reactive Controllers**

Advantages

- Fast
- Predictable
- Easy to debug

Problems

- **Difficult to scale**

Why Symbolic Planning?

Instead of programming every sequence manually:

```
IF A THEN  
  IF B THEN  
    IF C THEN  
      ...
```

The robot **reasons automatically** from:

1. Current world state
2. Goal description
3. Available actions

The planner generates the sequence **automatically**.

Planning Languages

Most robotic planners use:

PDDL

Planning Domain Definition Language

Two files:

File	Purpose
Domain	Defines actions, types, predicates (the "physics")
Problem	Defines objects, initial state, goals (a specific "puzzle")

PDDL Domain

Defines the **capabilities** of the system:

- Types
- Predicates
- Functions
- Actions

```
(:action move
  :parameters (?r ?from ?to)
  :precondition (and
    (at ?r ?from)
    (connected ?from ?to))
  :effect (and
    (not (at ?r ?from))
    (at ?r ?to)))
```

PDDL Problem

Contains **runtime** information:

- Objects
- Current predicates
- Goals

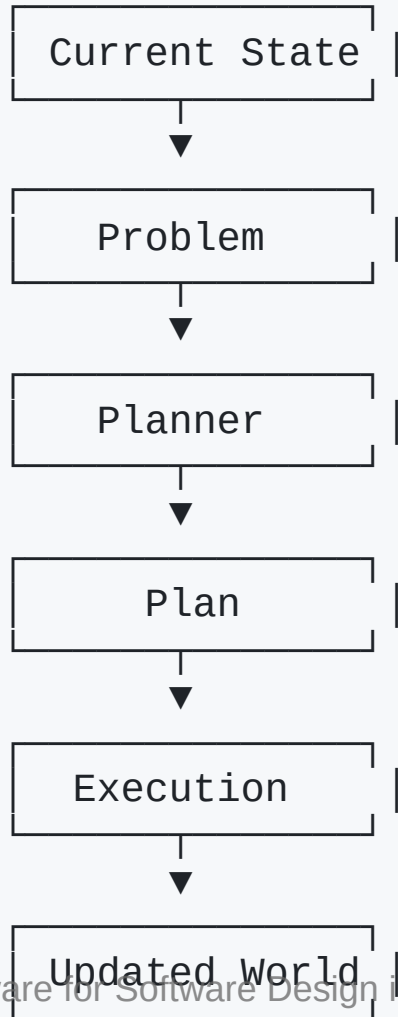
Initial State:

Robot at Kitchen
Cup at Kitchen

Goal:

Robot at LivingRoom
Cup at LivingRoom

Planning Pipeline



Before PlanSys2

The reference framework was:

ROSPlan

Limitations:

- ROS 1 only
- Single robot
- Limited execution model
- Less modular

PlanSys2 was designed as its **successor**.

Why ROS 2?

PlanSys2 takes advantage of ROS 2 features:

- DDS communication
- Lifecycle Nodes
- Better real-time support
- QoS profiles
- Improved reliability
- Better modularity

Main Goals of PlanSys2

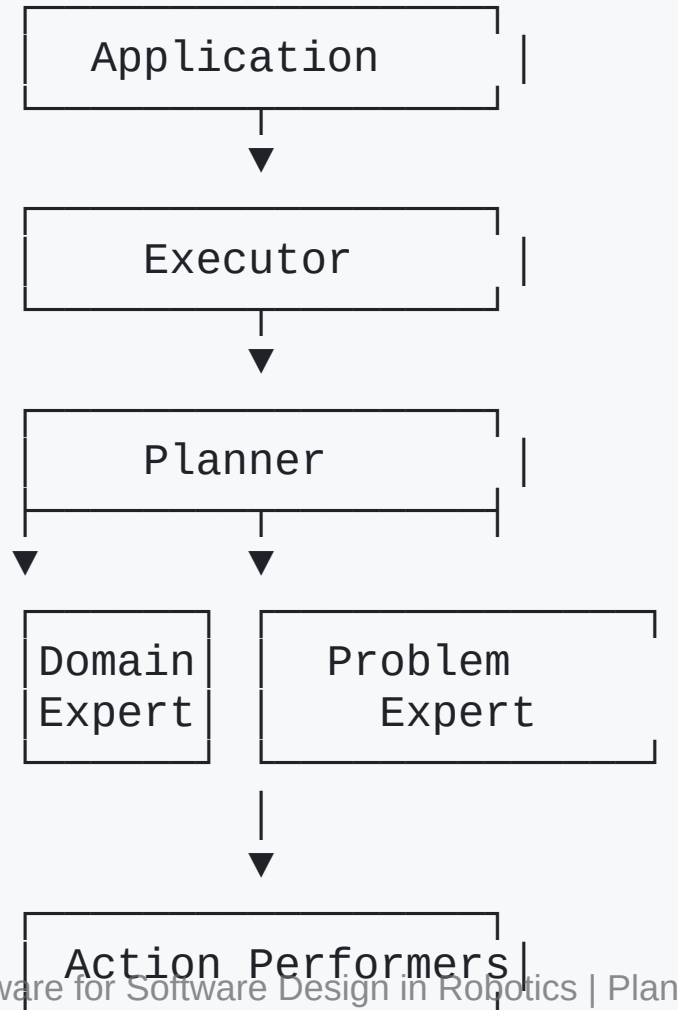
Goal	Description
Modular	Plug-and-play components
Extensible	Custom planners & performers
Efficient	Parallel execution via BTs
Multi-robot	Built-in multi-agent support
Explainable	Full execution trace published
ROS 2 Native	Lifecycle nodes, DDS, QoS

PlanSys2 Architecture

Main components:

1. **Domain Expert** — PDDL domain model
2. **Problem Expert** — Runtime state & goals
3. **Planner** — Generates plans
4. **Executor** — Runs plans via BTs
5. **Action Performers** — ROS 2 nodes that execute actions
6. **Application Controller** — Orchestrates the pipeline

Overall Architecture



Domain Expert

Stores:

- Domain model
- Actions
- Types
- Predicates

Responsibilities:

- Validate PDDL
- Answer domain queries
- Merge multiple domains

Problem Expert

Stores **runtime knowledge**:

```
(robot rb1)
(at rb1 kitchen)
(holding rb1 cup)
```

Responsibilities:

- Objects
- Predicates
- Functions
- Goals

Planner

Responsibilities:

- Receive planning request
- Build PDDL problem
- Invoke planner
- Return plan

Supported planners:

- POPF (temporal)
- TFD (temporal)

Additional planners can be added as **plugins**.

Executor

Goal



Requests plan



Transforms plan



Behavior Tree



Executes actions

Why Behavior Trees?

Behavior Trees provide:

Benefits

- Reactivity
- Parallel execution
- Recovery
- Better modularity
- Easier debugging

PlanSys2 converts every generated plan into a **BT**.

Plan Execution

Planner returns:

```
Move  
Pick  
Transport  
Drop
```

Executor transforms into:

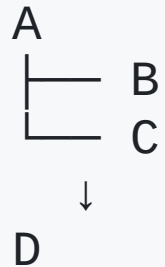
```
Behavior Tree  
└─ Sequence  
    ├── Move  
    ├── Pick  
    ├── Transport  
    └─ Drop
```

Benefits of BT Execution

Instead of executing **strictly sequentially**:

A → B → C

PlanSys2 **detects dependencies**:



Parallel actions reduce execution time.

Execution Flow Analysis

PlanSys2 analyzes:

- **Dependencies** between actions
- **Independent** actions that can run in parallel
- **Synchronization** points

Result: Maximum parallelism without violating constraints.

Action Lifecycle

Each action executes:

1. **Check** preconditions
2. **Apply** start effects
3. **Execute** action
4. **Monitor** conditions
5. **Apply** final effects

This closely follows **PDDL semantics**.

Action Performers

PlanSys2 does **not** execute robot code directly.

Instead, each action has one or more:

Action Performers

Examples:

- Move
- Pick
- Speak
- Navigate

Action Auction Protocol

Executor



Broadcast request



Available performers reply



Executor selects one



Execution begins

Advantages:

- Dynamic selection
- Multi-robot support
- Specialization

Multi-Robot Planning

Several robots may implement the same action:

```
(move rb1 kitchen living-room)
(move rb2 office kitchen)
```

The **executor** automatically assigns actions to the appropriate robot.

Explainability

PlanSys2 continuously publishes:

- Plans
- Action execution status
- Feedback
- Knowledge updates
- Auction messages

Useful for:

- Debugging
- Visualization
- Human supervision

Supported Applications

Domain	Examples
Service robots	Coffee delivery, visitor guidance
Warehouse robots	Pick-and-place, inventory
Agriculture	Crop inspection, harvesting
Inspection	Facility patrol, anomaly detection
Domestic robots	Cleaning, cooking assistance
Multi-robot systems	Coordinated team tasks

Example Planning Cycle

Goal: Serve Coffee



Planner



Navigate
Pick Cup
Navigate
Serve



Executor



Robot

Advantages

Feature	Status
ROS 2 Native	✓
Modular Architecture	✓
Plugin-based Planners	✓
Behavior Tree Execution	✓
Multi-robot Support	✓
Explainability	✓
Parallel Execution	✓

Limitations

Current limitations:

- Partial PDDL support
- Mainly POPF and TFD planners
- State consistency after failures
- Dynamic plan improvement still limited

Experimental Results

Simulation

- 1–3 robots
- Hundreds of plans
- Thousands of actions
- Zero execution failures

Real Robot (TIAGo)

- Two-hour experiment
- Zero system failures

Demonstrates **robustness** and **efficiency**.

Typical PlanSys2 Workflow

```
graph TD; A[Design PDDL] --> B[Load Domain]; B --> C[Update Knowledge]; C --> D[Set Goal]; D --> E[Generate Plan]; E --> F[Execute]; F --> G[Monitor]; G --> H[Repeat];
```

Design PDDL
▼
Load Domain
▼
Update Knowledge
▼
Set Goal
▼
Generate Plan
▼
Execute
▼
Monitor
▼
Repeat

Key Takeaways

1. Planning generates tasks **automatically**
2. PDDL separates domain from runtime knowledge
3. PlanSys2 integrates planning into **ROS 2**
4. Execution uses optimized **Behavior Trees**
5. Multi-robot execution is **built-in**
6. Modular architecture enables **reuse** and **extension**

Discussion

Questions for reflection:

- When is planning preferable to FSMs?
- Why execute plans as Behavior Trees?
- How does PlanSys2 support multiple robots?
- What happens when an action fails?
- Which robotic applications benefit most from symbolic planning?

References

Martín et al.

PlanSys2: A Planning System Framework for ROS2

IEEE Conference Paper

Primary source for this lecture.

Additional resources:

- [PlanSys2 Documentation](#)
- [ROS 2 Planning Examples](#)